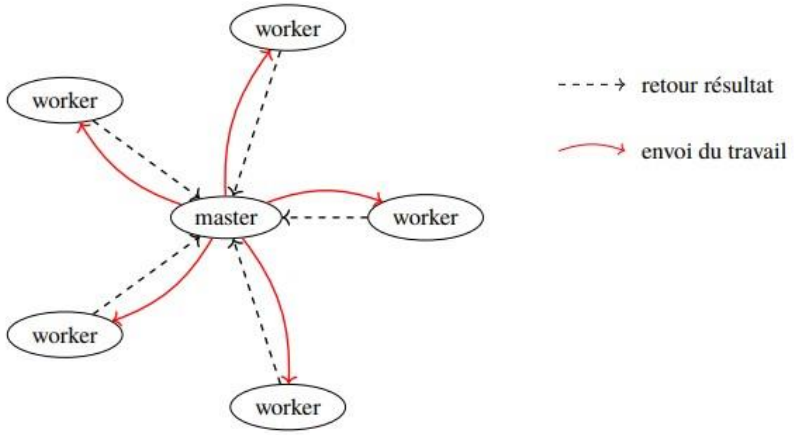
PROJET Programmation Parallèle : MPI & OPENMP

Partie A : MPI - Modèle Master/Worker

Contexte :

On souhaite paralléliser le calcul de la somme d'un tableau de 100 entiers.

Le modèle utilisé est le modèle Master/Worker.



Principe :

- Le tableau est divisé en plusieurs parties.
- Chaque processus worker calcule une somme partielle.
- Le processus master distribue les données et collecte les résultats.

Organisation :

- 4 processus workers (P0, P1, P2, P3).
- 1 processus master.
- Chaque worker traite une portion du tableau..
- Le master calcule la somme finale.

Travail demandé :

1. Implémenter le modèle Master/Worker avec MPI.

2. Tester avec plusieurs tailles de tableau.
3. Mesurer les performances.

Partie B : Fibonacci MPI + OpenMP

Objectifs :
- Utiliser MPI et OpenMP ensemble.
- Étudier le parallélisme hybride.

Travail demandé :
1. Implémenter Fibonacci (séquentiel, OpenMP, MPI).
2. Ecrire la version parallèle en combinant MPI + OpenMP.
3. Étudier les performances (speedup, efficacité).

Partie C : Parallélisme de boucle

1 – On considère un tableau de n cases contenant chacune une valeur aléatoire :

a. Écrire un programme C qui lit la taille du tableau sur la ligne de commande, le crée et le remplit de valeurs aléatoires.

b. Écrire une méthode carre(...) prenant le tableau en paramètre et élevant la valeur de chacune des cases au carré.

c. Modifiez ce programme pour que les boucles « for » soient parallélisées par OpenMP.

d. Faites afficher par votre programme le nombre de threads qui sont créés par OpenMP. Que remarquez-vous ?

e. Modifiez votre programme pour fixer le nombre de threads de la boucle for à 1, 5 puis 10. Dans chaque cas, mesurez le temps pris par votre programme, que constatez-vous ?

f. Cherchez comment à l'aide de la variable OMP_NUM_THREADS il est possible de modifier le nombre de threads sans modifier votre code.

6 – On veut profiter de la boucle de la méthode carre() pour faire la somme de tous les éléments du tableau.

a. Dans la méthode carre(...) précédente, déclarez une variable *int total* avant la boucle « for » et utilisez là pour faire la somme de toutes les cases du tableau dans la boucle. Que vaut total en sortie de boucle ? pourquoi ?

b. Indiquez maintenant que total est une variable privée. Que vaut-elle en sortie de boucle ? pourquoi ?

c. À l'aide d'une section critique, implémentez correctement la somme des éléments. Que constatez-vous en terme de performances ?

d. À l'aide d'une opération atomique, modifiez votre implémentation. Que constatez-vous par rapport à la version précédente ?

e. Utilisez la « clause de réduction » pour arriver au même résultat et discutez les mérites de cette solution comparée à la précédente

Livrables :
- Code source
- Rapport
- Résultats expérimentaux